Submitted By: Jiya Singla **Submission Date**: 05 July 2026

Week 7 Celebal Assignment

Delta Lake MERGE Implementation

1. INTRODUCTION

Data engineering pipelines often receive new and updated data continuously. Processing the entire dataset every time new data arrives is inefficient and time-consuming. To solve this problem, organizations use **incremental data processing**, where only the changed or newly arrived records are processed.

Delta Lake is an open-source storage layer that extends data lakes with powerful features such as ACID transactions, schema enforcement, data versioning, time travel, and efficient MERGE operations. One of its most important capabilities is the MERGE command, which combines INSERT, UPDATE, and DELETE operations into a single statement.

This assignment demonstrates the implementation of Delta Lake MERGE using the Superstore Dataset in Databricks.

2. OBJECTIVE

The objectives of this assignment were:

- Load the Superstore dataset into a Delta table.
- Perform basic data cleaning.
- Create an incremental dataset to simulate new business data.
- Apply the Delta Lake MERGE operation.
- Validate the updated dataset.
- Understand incremental ETL processing using Delta Lake.

3. DATASET DESCRIPTION

The **Superstore Dataset** is a retail sales dataset containing customer orders from different regions.

Dataset Details

Submitted By: Jiya Singla **Submission Date**: 05 July 2026

Attribute	Description
Dataset Name	Superstore Dataset
Source	Kaggle
Format	CSV
Total Records	9,994
Total Columns	21

Important Columns

- Row ID
- Order ID
- Order Date
- Customer ID
- Customer Name
- Segment
- City
- State
- Region
- Product ID
- Category
- Sub-Category
- Product Name
- Sales
- Quantity
- Discount
- Profit

This dataset represents customer purchases made across different regions and product categories.

4. TOOLS AND TECHNOLOGIES USED

The following technologies were used during this assignment:

- Databricks Community Edition
- Apache Spark (PySpark)
- Delta Lake
- Python
- CSV Dataset
- GitHub

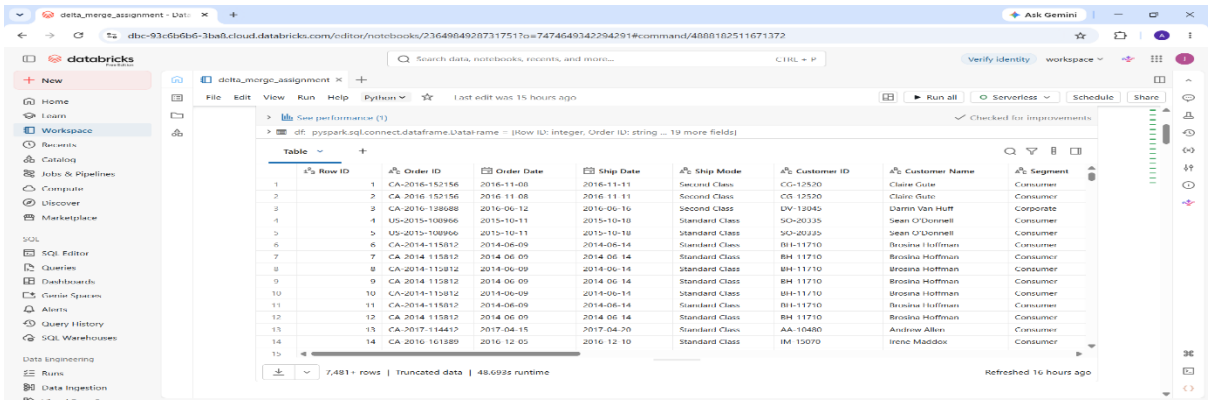
5. METHODOLOGY

Step 1: Loading the Dataset

The Superstore CSV dataset was uploaded to Databricks and loaded into a Spark DataFrame using PySpark.

Activities performed:

- Uploaded CSV file
- Read dataset using Spark
- Inferred schema
- Displayed dataset



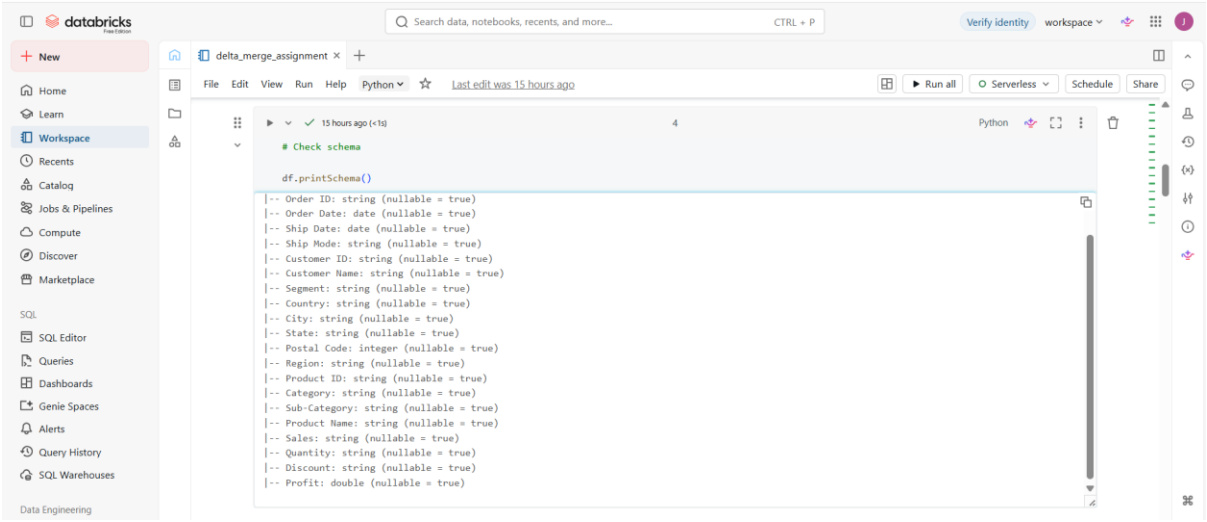
Submitted By: Jiya Singla **Submission Date**: 05 July 2026

Step 2: Dataset Exploration

The dataset was explored to understand its structure.

The following checks were performed:

- Displayed schema



- Counted total rows
- Counted total columns
- Displayed sample records

This ensured that the dataset had been loaded successfully.

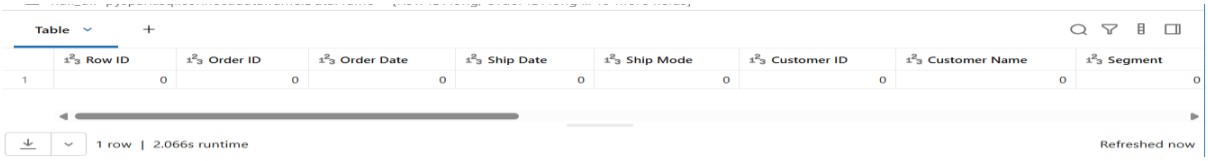
Step 3: Data Cleaning

Data quality is essential before processing.

The following cleaning operations were performed:

- Checked for missing values.
- Removed duplicate records.
- Verified the cleaned dataset.

The dataset contained no missing values, and duplicate records were removed successfully.



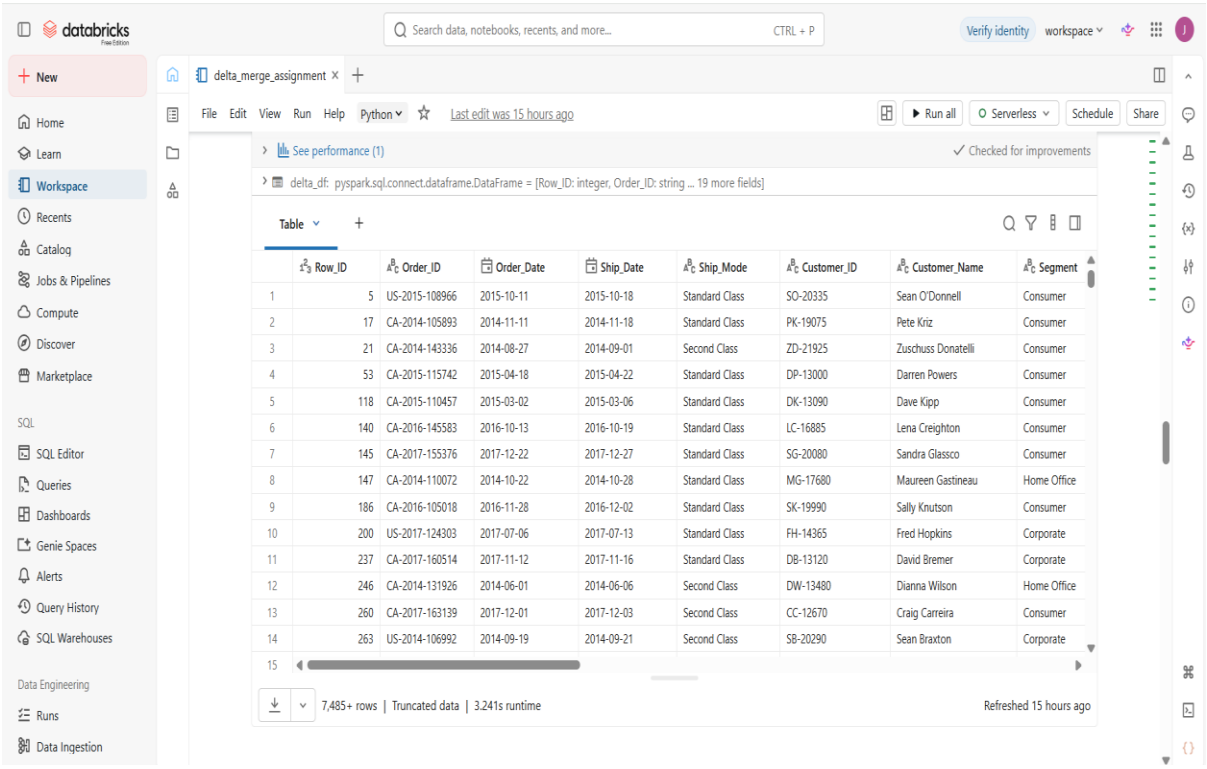
Step 4: Creating the Delta Table

The cleaned DataFrame was converted into a Delta table.

Delta tables provide several advantages:

- ACID Transactions
- Schema Enforcement
- Data Versioning
- Better Performance
- Reliable Incremental Processing

This Delta table served as the target table for the MERGE operation.



Step 5: Creating the Incremental Dataset

To simulate real-world incremental data arrival, a second dataset named **superstore_incremental.csv** was created.

The incremental dataset contained:

- One existing order with updated values.

- One completely new order.

This dataset was later merged into the Delta table.


incremental_df: pyspark.sql.connect.dataframe.DataFrame = [Order ID: string, Customer ID: string ... 4 more fields]

Table +

	Order ID	Customer ID	Customer Name	Sales	Quantity	Profit
1	CA-2016-152156	CG-12520	Claire Gute	350	2	40
2	NEW-2026-1000...	AB-10001	John Smith	500	3	70

2 rows | 0.771s runtime Refreshed now

Step 6: Applying the MERGE Operation

The Delta Lake MERGE command was executed using **Order ID** as the matching key.

The MERGE operation performed two actions:

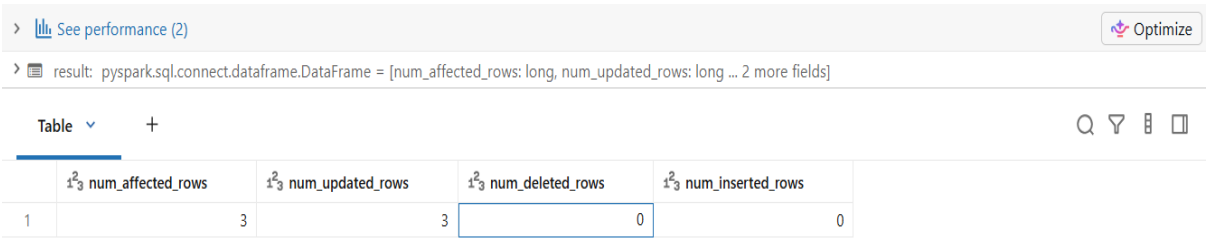
Update Existing Record

The existing order **CA-2016-152156** was updated with new sales, quantity, and profit values.

Insert New Record

A new order **NEW-2026-100001** was inserted into the Delta table.

This demonstrates how Delta Lake handles incremental updates efficiently.


See performance (2) Optimize

result: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_updated_rows: long ... 2 more fields]

Table +

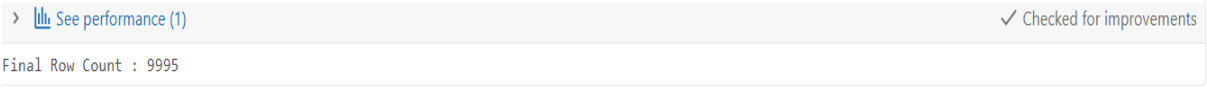
	num_affected_rows	num_updated_rows	num_deleted_rows	num_inserted_rows
1	3	3	0	0

Step 7: Validation

The final Delta table was validated using the following checks:

- Displayed updated Delta table.
- Verified total row count.
- Checked for duplicate Order IDs.
- Confirmed successful update and insert operations.

The validation confirmed that the MERGE operation worked correctly.



6. RESULTS

The assignment successfully demonstrated:

- Loading data into Spark.
- Performing data cleaning.
- Creating a Delta table.
- Simulating incremental data.
- Updating existing records.
- Inserting new records.
- Validating the final dataset.

The final Delta table accurately reflected both updated and newly inserted records.

7. ADVANTAGES OF DELTA LAKE MERGE

Using Delta Lake MERGE provides several benefits:

- Combines INSERT and UPDATE into a single operation.
- Supports ACID-compliant transactions.
- Prevents inconsistent updates.
- Efficiently processes incremental data.
- Improves ETL pipeline performance.
- Simplifies data warehouse maintenance.

8. REAL-WORLD APPLICATIONS

Delta Lake MERGE is widely used in industries such as:

- E-commerce Order Management

- Banking Transactions
- Customer Relationship Management (CRM)
- Inventory Management
- Healthcare Record Management
- Employee Information Systems
- Enterprise Data Warehousing

9. CHALLENGES FACED

During implementation, the following challenges were encountered:

- Uploading datasets into Unity Catalog volumes.
- Understanding Delta table creation.
- Creating an incremental dataset.
- Resolving column naming differences (Order ID vs Order_ID) during the MERGE operation.
- Validating successful updates and inserts.

These issues were resolved by using the correct column names and verifying outputs after each step.

10. LEARNING OUTCOMES

After completing this assignment, I gained practical knowledge of:

- Reading CSV files using PySpark.
- Working with Spark DataFrames.
- Performing data cleaning operations.
- Creating Delta tables.
- Implementing Delta Lake MERGE.
- Validating incremental data processing.
- Understanding how MERGE supports modern ETL pipelines.

11. CONCLUSION

This assignment provided hands-on experience with Delta Lake MERGE for incremental data processing in Databricks. The implementation demonstrated how existing records can be updated and new records inserted efficiently using a single MERGE statement. The assignment also highlighted the importance of data cleaning, validation, and Delta Lake features such as ACID transactions and schema enforcement. Overall, the project successfully achieved all the stated objectives and provided practical exposure to modern data engineering workflows.